

Warehouse Operations Planning: Final Report

Amonios Beshara, Hazem Nasr, Mikhael Khalil, and Yehia El Hodeiby

May 16, 2026

Title and Team Roles

Roles:

- Thread A (Human): Yehia (enhanced algorithm design), Hazem (implementation and complexity analysis)
- Thread B (AI-assisted): Amonios (prompting and reasoning-audit lead), Mikhael (AI-assisted coding, testing, and verification)
- Integration and editing: Hazem (repo formatting and organization), Amonios (report integration and editing)

Contribution Matrix: Note on contributions. Much of the work was done locally while sitting

Task	Weight (%)	Amonios (%)	Mikhael (%)	Hazem (%)	Yehia (%)
Problem modeling	10	15	15	30	40
Algorithm design (Thread A)	25	0	0	35	65
AI-assisted workflow (Thread B)	20	60	40	0	0
Implementation (C++ code)	20	10	35	40	15
Testing and outputs	15	20	40	25	15
Report integration and editing	10	40	10	40	10

Table 1: Contribution matrix.

together, so the repo history does not fully reflect effort. Hazem had less algorithm-design work than Yehia but was responsible for formatting and organizing the repo to balance the workload.

Formal Problem Definition

We must construct a feasible next-day warehouse operations plan. Inputs include a warehouse graph (dispatch node D, optional supplier node V, shelves), items with stock and handling constraints, customer orders with release times, deadlines, priorities, and packing times, picker shifts and capacity limits, and optional restock constraints (reserve, prep time, delivery time, and max delivery weight). The plan must assign orders to picker batches, schedule any needed restocks, and route each batch through shelves to meet deadlines and shift constraints while respecting stock feasibility, perishable handling limits, and routing feasibility. If the plan is feasible, output restock requests and batches with times and routes; otherwise report INFEASIBLE with a specific reason.

Thread A: Human Algorithm Development

Problem model. The team modeled the warehouse as a weighted undirected graph and used all-pairs shortest paths for routing. Orders are batched per picker and routed by a nearest-neighbor heuristic. Feasibility is enforced through stock, capacity, release, shift, deadline, and perishable-time checks. **Algorithm overview.** The human thread implemented two greedy strategies:

- **Baseline greedy:** orders are sorted by earliest deadline and restocking is triggered only when stock runs out.
- **Enhanced greedy:** orders are scored by a weighted linear function of normalized features (priority, slack, stock urgency, expected handling time, and price). Restocks are scheduled with lead time, including buffers for high delay risk.

Batching uses a composite rule that selects the next order by maximizing $\text{score} - \lambda \cdot \text{normalizedDistance}$ with $\lambda = 0.2$, and routes are built by a nearest-neighbor tour over required shelves. **Correctness (feasibility) sketch.** Each batch is accepted only if stock is available (after scheduled restocks), weight capacity is satisfied, release and shift windows are respected, deadlines are met, and perishable max-wait is not exceeded. Therefore, any output plan is feasible by construction when the algorithm succeeds. **Complexity (high-level).** Let V be the number of nodes, M the number of orders, P the number of pickers, and S the number of shelves per batch. The dominant preprocessing cost is Floyd–Warshall $O(V^3)$. Sorting orders is $O(M \log M)$, batch construction is $O(M^2)$, and routing is $O(S^2)$ per batch. **Thread A test results.**

Case	Baseline Feasible	Enhanced Feasible	Baseline Restocks	Enhanced Restocks	Runtime (ms)
1	Yes	Yes	0	0	18.58
2	Yes	Yes	3	2	13.58
3	No (unassigned)	Yes	0	8	39.13
4	No (VIP deadline)	No (VIP deadline)	0	0	4.60

Table 2: Thread A test results for cases 1–4.

Thread B: AI-Assisted Development

AI workflow summary. The AI-assisted thread implemented a priority-driven greedy planner with restock scheduling, batching, and feasibility validation. The algorithm phases are: (1) all-pairs shortest paths, (2) demand analysis, (3) restock scheduling with delay buffers, (4) order prioritization by priority and slack, (5) capacity-aware greedy batching, (6) greedy routing via nearest-neighbor TSP, and (7) feasibility validation. **Complexity (from code comments).** Floyd–Warshall $O(V^3)$, sorting $O(M \log M)$, batching $O(M \cdot P)$, routing $O(S^2)$ per batch, and validation $O(M \cdot S)$. Overall $O(V^3 + M \cdot P + B \cdot S^2)$ where B is the number of batches. **Thread B execution outputs. Thread B test notes.** Runs were deterministic (fixed seed = 42) and all four cases executed on the same machine within the 60-second limit. During development, cases 1–3 initially returned INFEASIBLE due to a batch-packing bug, which was fixed by a round-robin batch loop and restock-arrival guards. Manual edge cases tested included: zero-demand order (empty lines), single picker with one fully stocked order, exact reserve equals shortfall, reserve less than shortfall (INFEASIBLE), and overtime trigger by shrinking shift end time. **Human edits and corrections.** We revised AI output in four main places: (1) restock splitting by max delivery weight, (2) priority-aware order sorting (VIP and urgent tiers with deadline tie-breaks),

Case	Feasible	Restock Requests	Runtime (ms)
1	Yes	0	6.32
2	Yes	2	10.17
3	Yes	8	34.19
4	No (VIP deadline)	[N/A]	4.98

Table 3: Thread B execution summary for cases 1–4.

(3) perishable shelf-wait checks, and (4) round-robin batch packing so orders were not assigned to a single picker. These edits fixed shallow scheduling assumptions and reduced false confidence in feasibility.

Good Prompts, Weak Prompts, and Bad Prompts

Discussion lead: Thread B.

Prompt #	Prompt summary	Good/Weak/Bad	Why?	AI output	Understanding?	Productivity?	Risk
1	Initial framing: design algorithm from vague prompt	Bad	Too broad; missing constraints	Generic greedy scheduler	No	Medium	High
2	Graph model and Floyd-Warshall complexity	Good	Specific graph and request	Correct F-W and $O(V^3)$	Yes	Medium	Low
3	Order sorting to minimize lateness	Weak	Missing priority tiers	EDF only, incomplete	Partial	Medium	Medium
4	Restock scheduling code (no context)	Bad	Missing constraints	Stub with three bugs	No	Medium	High
5	Multi-delivery split with max weight	Good	Exact constraints given	Correct splitting loop	Yes	Medium	Low
6	Nearest-neighbor TSP heuristic	Good	Clear start/stop/return	Correct NN, $O(S^2)$	Yes	Medium	Low
7	Perishable handling question	Weak	Too vague	Generic expiry advice	No	Low	Medium
8	Batch packing bug with code snippet	Good	Specific bug and code	Round-robin fix	Yes	Medium	Low
9	Infeasibility reporting	Good	Clear failure modes	Structured reasons	Yes	Medium	Low
10	Case 2 stock violation debug	Good	Failure time and code	Guard on restock arrival	Yes	Medium	Low

Table 4: Required prompt quality table.

Prompt quality summary. We documented 12 interactions: 5 good, 3 weak, and 4 bad prompts. The good prompts always included constraints or code context; the bad prompts asked for generation without context and produced plausible but wrong output. **AI as reviewer vs. generator.** The most reliable use of AI was as a reviewer after we had a design. When we asked it to generate without context, it introduced subtle bugs that took more time to fix than writing the logic ourselves.

Getting the Answer Right vs. Understanding the Answer

We evaluate understanding using the following checks (complete at least four):

- Explain the algorithm without reading AI output: Achieved for Thread A; partial for Thread B.
- Trace the algorithm on a new example: Achieved with one new small case per thread.
- Derive time complexity step-by-step: Achieved for both threads.
- Explain why the greedy choice is valid: Partial; relied on empirical justification.
- Identify an edge case where a naive solution fails: Achieved (VIP deadline case).
- Modify the algorithm under a new constraint: Achieved (restock max weight constraint).
- Explain why the C++ implementation matches the pseudocode: Achieved after review.
- Answer oral questions about the algorithm and code: Achieved for main components.

Completion sentence:

AI helped us get an answer in a fast, structured way, but understanding required manual tracing, constraint checks, and corrections.

Productivity Comparison

Time logs. Summary metrics.

Student	Date	Task	AI used?	Time spent	Output produced
Yehia	2026-04-22	Enhanced algorithm design	No	6 h	Scoring model and batch rule
Hazem	2026-04-23	C++ implementation and complexity	Yes	5 h	Thread A code and analysis
Mikhael	2026-04-24	AI-assisted coding and tests	Yes	4 h	Thread B code and run logs
Amonios	2026-04-25	Prompt audit and report writing	Yes	4 h	Prompt log and analysis notes

Table 5: Productivity time log.

Measurement	Human thread	AI-assisted thread	Interpretation
Time to first problem model	60 min	30 min	AI sped drafting but missed constraints
Time to first algorithm idea	90 min	40 min	Faster with AI, weaker justification
Time to first pseudocode	2.5 h	1.5 h	AI accelerated structure
Time to first working C++ code	6 h	3 h	AI sped coding, required review
Number of major revisions	2	4	AI output needed more correction
Number of edge cases discovered	3	4	AI suggested more cases
Number of incorrect claims found	1	3	AI reasoning was shallow
Number of failed tests	1	3	Early AI batches failed feasibility
Time spent debugging or correcting	2 h	3.5 h	AI saved time early, cost time later
Final confidence in correctness	High	Medium	Confidence tied to manual verification

Table 6: Productivity comparison summary.

Correctness and Complexity Comparison

Correctness evidence. Thread A and Thread B both produced feasible plans for cases 1–3 and identified the infeasible VIP deadline in case 4. Thread A reported a baseline failure in case 3 while the enhanced variant succeeded. Thread B required manual fixes for restock splitting, priority ordering, perishable checks, and round-robin batching to avoid incorrect feasibility. **Complexity comparison. Interpretation.** Both approaches share a $O(V^3)$ preprocessing bottleneck. Thread

Thread	Stated Complexity
Thread A (Human)	$O(V^3 + M^2 + S^2)$ with $O(V^3)$ dominating
Thread B (AI-assisted)	$O(V^3 + M \cdot P + B \cdot S^2)$

Table 7: High-level complexity comparison.

A’s enhanced scoring can push toward $O(M^2)$, while Thread B trades for $O(M \cdot P)$ but depends more on validity checks. Complexity claims were similar, but correctness depended on careful constraint handling.

When AI Helped (Concrete Examples)

Provide at least three examples from the group work. For each, state the impact.

1. AI suggested a phased planner structure that matched the problem constraints.
Impact: Clarity and productivity.
2. AI generated edge-case ideas (VIP deadline infeasible case) that we adopted.
Impact: Testing and correctness.
3. AI accelerated C++ scaffolding for data structures and output formatting.
Impact: Productivity.

When AI Harmed or Could Have Harmed (Concrete Examples)

Provide at least three examples from the group work.

1. AI initially ignored restock weight limits, which would allow infeasible deliveries.
2. AI provided a shallow correctness argument that masked missing constraints.
3. AI suggested batching by capacity only, which risks deadline violations.
4. AI missed perishable window checks until prompted with exact constraints.

Recommendation for Future CSCE 2202 Students

Use AI after an initial human attempt. Start with manual problem modeling and at least one baseline algorithm, then use AI to review constraints, generate edge cases, and scaffold code. Require manual verification of correctness and complexity claims before accepting any AI-generated reasoning.

Final Message on AI Usage and the Core Project Question

AI helps students and professionals work faster when it is used as a reviewer or testing assistant after the core reasoning is done. It weakens algorithmic skills when it substitutes for modeling, proof thinking, or manual tracing. In academic settings, the cost of shallow understanding is higher because those skills are the learning outcomes; in professional settings, speed gains are acceptable only if verification practices are strong.

Appendix A: Source Code and Execution Logs

- Human solution code: https://github.com/HazeMOONium/Analysis-Project/blob/main/human_solution.cpp (local file: `human_solution.cpp`)
- AI-assisted solution code: https://github.com/HazeMOONium/Analysis-Project/blob/main/ai_solution.cpp (local file: `ai_solution.cpp`)
- Human execution output: https://github.com/HazeMOONium/Analysis-Project/tree/main/human_output (local file: `human_output/output.txt`)
- AI execution output cases 1–4: https://github.com/HazeMOONium/Analysis-Project/tree/main/ai_output (local files: `ai_output/execution_output_case_*.txt`)

Appendix B: AI Usage Appendix

- AI tool used: GitHub Copilot Chat (GPT-5.2-Codex)
- Dates of AI use: 2026-05-14 to 2026-05-16
- Prompts: Summarized in the 10-row prompt log above
- AI outputs: Algorithm outline, C++ scaffolding, test ideas, debugging suggestions
- Accepted by group: Phase structure, NN TSP, infeasibility reporting
- Rejected by group: EDF-only sorting, restock stubs without max weight, vague perishable advice
- Corrected by group: Restock splitting, priority tiers, perishable constraint, round-robin batching

Important Note: Yehia is inteding to do the bonus part.